Niveles 1, 2 y 3 · Guía del problema base

QuickDrop: *el plan de reparto*

Tu equipo es una consultora que desarrolla software para QuickDrop, una empresa de logística de última milla. Cada mañana reciben pedidos, una flota y la ubicación del depósito, y necesitan un plan de reparto. No se busca la solución óptima: se busca una estrategia razonable, justificada y bien diseñada.

1 Los niveles

Nivel	Perfil	Requisitos	Eventos
1 · Primeros pasos	Programación básica, estructuras de datos	Se espera C con `struct`; vale cualquier lenguaje. CSV o JSON.	No tiene
2 · Intermedio	Resolución de problemas, JSON	JSON, Git y GitHub recomendados	Opcionales
3 · Avanzado	Reglas que cambian, calidad, arquitectura	JSON, Git y GitHub obligatorios	Obligatorios

Un equipo del Nivel 1 que quiera los eventos puede anotarse en el Nivel 2, o en el 3 si cumple sus requisitos. Cada nivel tiene su propio ranking. **Se permite usar IA**: lo difícil no es escribir el algoritmo, sino las decisiones de diseño y poder defenderlas.

2 El mundo

- Una grilla de **20 × 20** cuadras; coordenadas enteras de 0 a 19.
- Distancia **Manhattan**: `|x1 - x2| + |y1 - y2|` cuadras.
- Todos los vehículos salen del **depósito** a la `hora_salida` y terminan en el depósito (la vuelta final no se escribe en el plan).
- Tiempos en **minutos enteros**; horas como `"HH:MM"`.

> **Leé todo del archivo `mundo`.** El depósito, los costos, la velocidad y los tiempos pueden cambiar de un dataset a otro, **incluida la ubicación del depósito**. Un programa que suponga el depósito en (0,0) va a perder puntos.

3 Los datos de entrada

Cada dataset trae tres archivos, en **JSON y en CSV** (los mismos campos): `mundo`, `pedidos` y `vehiculos`. La especificación completa está en el repo, en `ESPECIFICACION.md`.

mundo

```
{ "grilla_ancho": 20, "grilla_alto": 20,
  "deposito_x": 0, "deposito_y": 0,
  "hora_salida": "08:00",
  "tiempo_servicio": 5, "tiempo_carga": 10,
  "min_por_cuadra": 3,
  "costo_por_cuadra": 3,
  "costo_por_vehiculo": 60,
  "costo_por_minuto_total": 1,
  "costo_por_falla": 200 }
```

En CSV: `clave,valor`, una fila por campo.

pedidos

```
id,x,y,peso
1,15,8,12
2,6,14,4
```

vehiculos

```
id
V1
V2
```

Un campo opcional puede faltar, valer `null` o estar vacío. Si un archivo trae campos que tu programa no conoce, ignoralos.

4 Lo que entregás: el plan

Tu programa lee el dataset y escribe el plan: para cada vehículo, la lista ordenada de paradas, con un `id` de pedido o la palabra `DEPOSITO` (volver al depósito e iniciar un nuevo viaje).

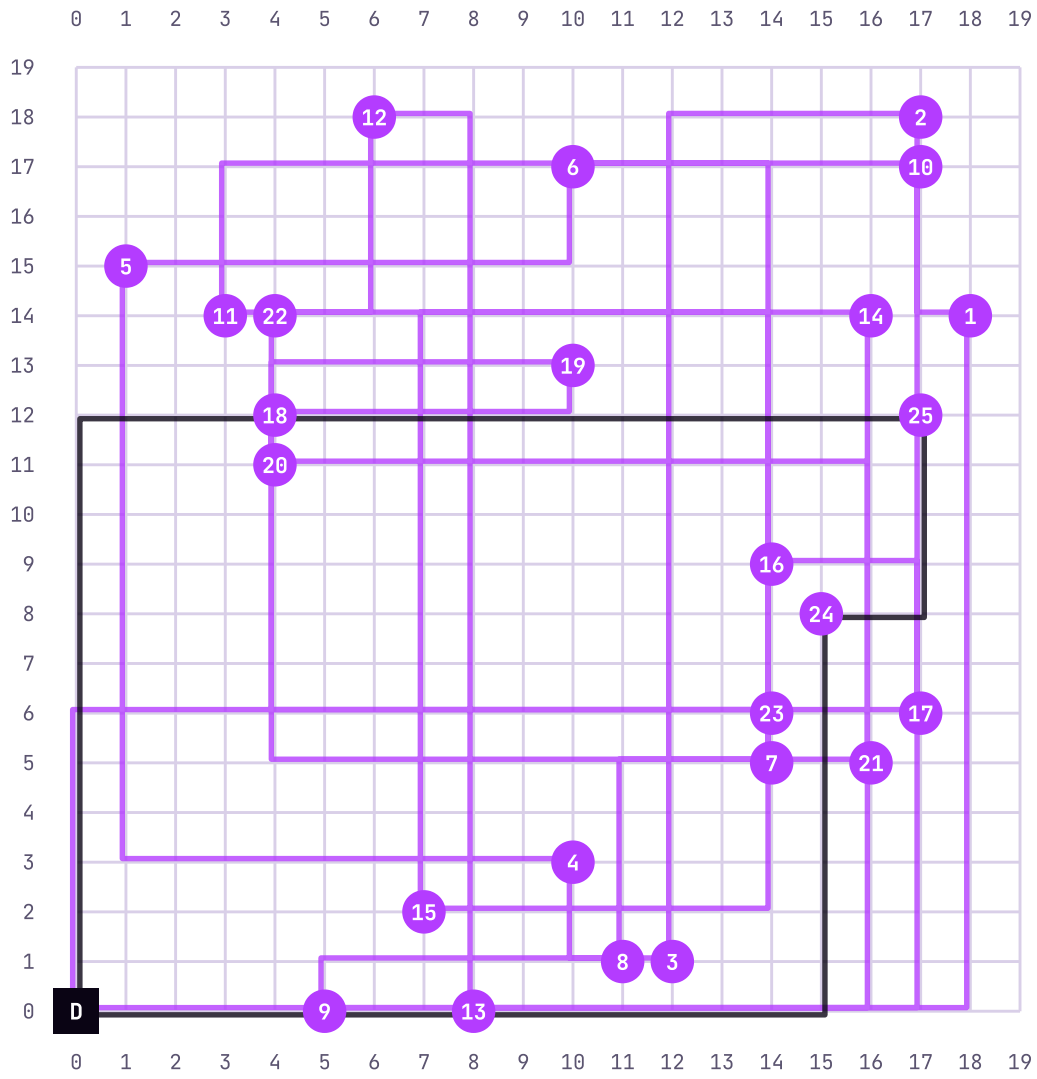
```
{ "rutas": [
  { "vehiculo": "V1",
    "paradas": [2, 1, 10, "DEPOSITO", 8, 3] },
  { "vehiculo": "V2", "paradas": [4, 9, 7] }
] }
```

```
vehiculo,parada
V1,2
V1,1
V1,10
V1,DEPOSITO
...
```

Un plan es **inválido** (vale 0 en ese dataset) si usa un vehículo que no existe, repite un vehículo, incluye un pedido que no existe o repite un pedido. Un pedido que no aparece en el plan no lo invalida: **falla**.

5 Ejemplo de salida: una ruta *ineficiente*

Para que veas **exactamente el formato que esperamos**, este plan reparte un dataset de 25 pedidos con 2 vehículos: **V1 lleva 23 pedidos**, en orden de número, y **V2 lleva los 2 restantes**. Es válido, pero cruza la ciudad una y otra vez: **no lo copies**, usalo solo para ver la forma.



Violeta: el recorrido de V1. Negro: el de V2. Los dos salen del depósito D y vuelven a él.

El plan en JSON

```
{ "rutas": [
  {"vehiculo": "V1", "paradas": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
19, 20, 21, 22, 23] },
  { "vehiculo": "V2", "paradas": [24, 25] }
] }
```

El mismo plan en CSV

Una fila por parada, en orden; las filas de un mismo vehículo forman su ruta.

```
vehiculo,parada
V1,1
V1,2
V1,3
V1,4
V1,5
V1,6
V1,7
V1,8
V1,9
```

```
vehiculo,parada
V1,10
V1,11
V1,12
V1,13
V1,14
V1,15
V1,16
V1,17
V1,18
```

```
vehiculo,parada
V1,19
V1,20
V1,21
V1,22
V1,23
V2,24
V2,25
```

Lo que responde el simulador

```
Plan válido

COSTO TOTAL: 2641
  distancia  430 cuadras × 3 = 1290
  vehículos  2 usados (costo fijo) = 120
  tiempo     1231 min × 1 = 1231
  fallas     0 × 200 = 0

Entregas: 25/25 entregados · 0 sin asignar
```

El plan entrega todo, pero **cuesta caro**: tu programa tiene que encontrar uno mejor.

---

6 Cómo se simula y cuánto cuesta

Para cada vehículo, desde el depósito a la `hora_salida`: viajar a una parada tarda `cuadras × min_por_cuadra`; entregar ocupa `tiempo_servicio` minutos; una parada `DEPOSITO` tarda `tiempo_carga` minutos; al final vuelve al depósito.

```
costo = costo_por_cuadra       × distancia total (cuadras, todos los vehículos)
      + costo fijo             de cada vehículo usado
      + costo_por_minuto_total × makespan (minutos hasta que vuelve el último vehículo)
      + costo_por_falla        × pedidos que fallan
```

Menor es mejor. Hay más vehículos de los estrictamente necesarios: decidir cuántos usar es parte del problema.

---

7 El simulador

El mismo código que usamos para evaluarte. El costo que te muestra es exactamente el que cuenta.

- **En tu máquina** (Python 3.9 o más, sin dependencias):
  `https://github.com/clubProgramacionUNViMe/quickdrop-sim`
  `python -m quickdrop simular datos/ejemplo datos/ejemplo/plan_ejemplo.json --detalle`
- **En la web:** `https://dygd5d0drje17.cloudfront.net/simulador` — reglas, datasets de práctica y entregas.

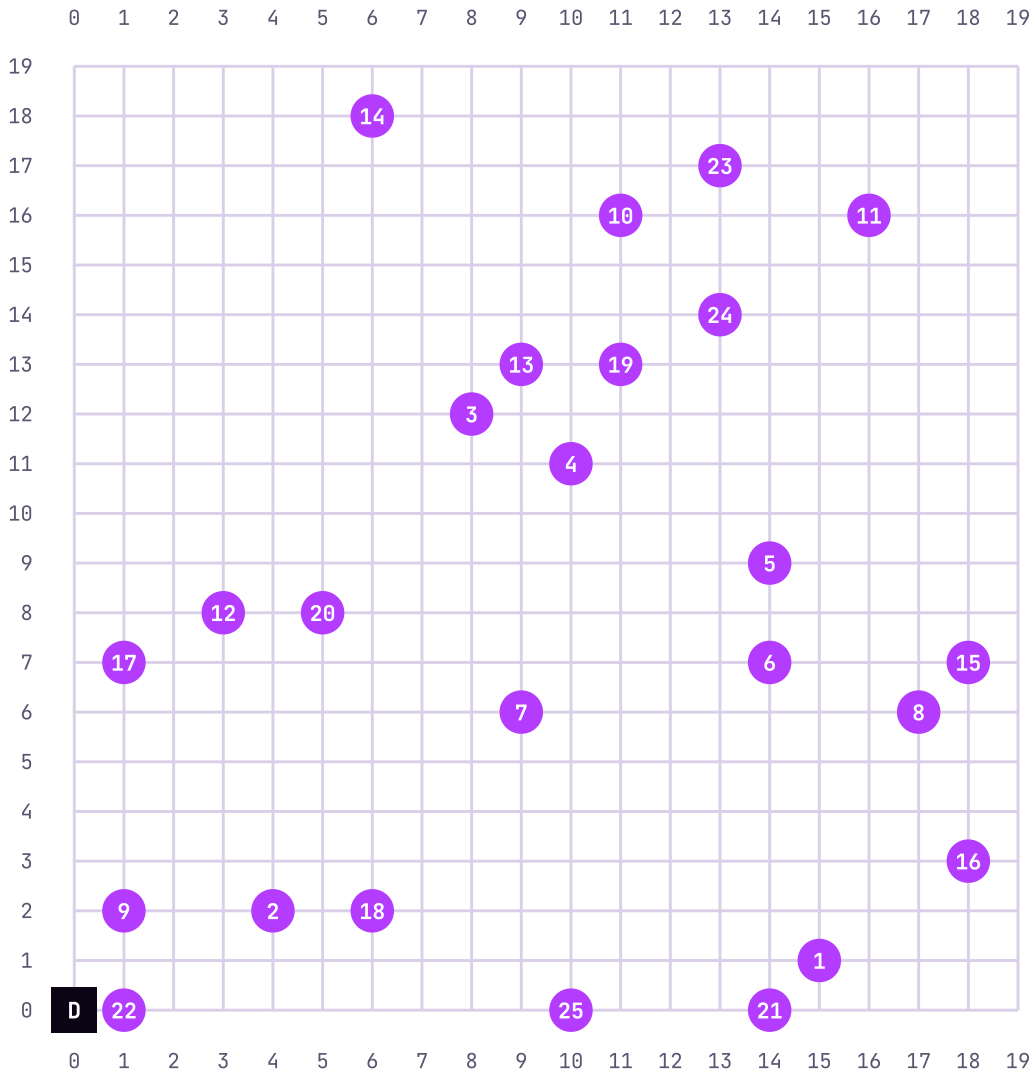
---

8 Tu ejercicio de práctica

Estos son los datasets de práctica de cada nivel, dibujados: el depósito es la D y cada círculo, un pedido. Los de evaluación son distintos, pero del mismo tamaño.

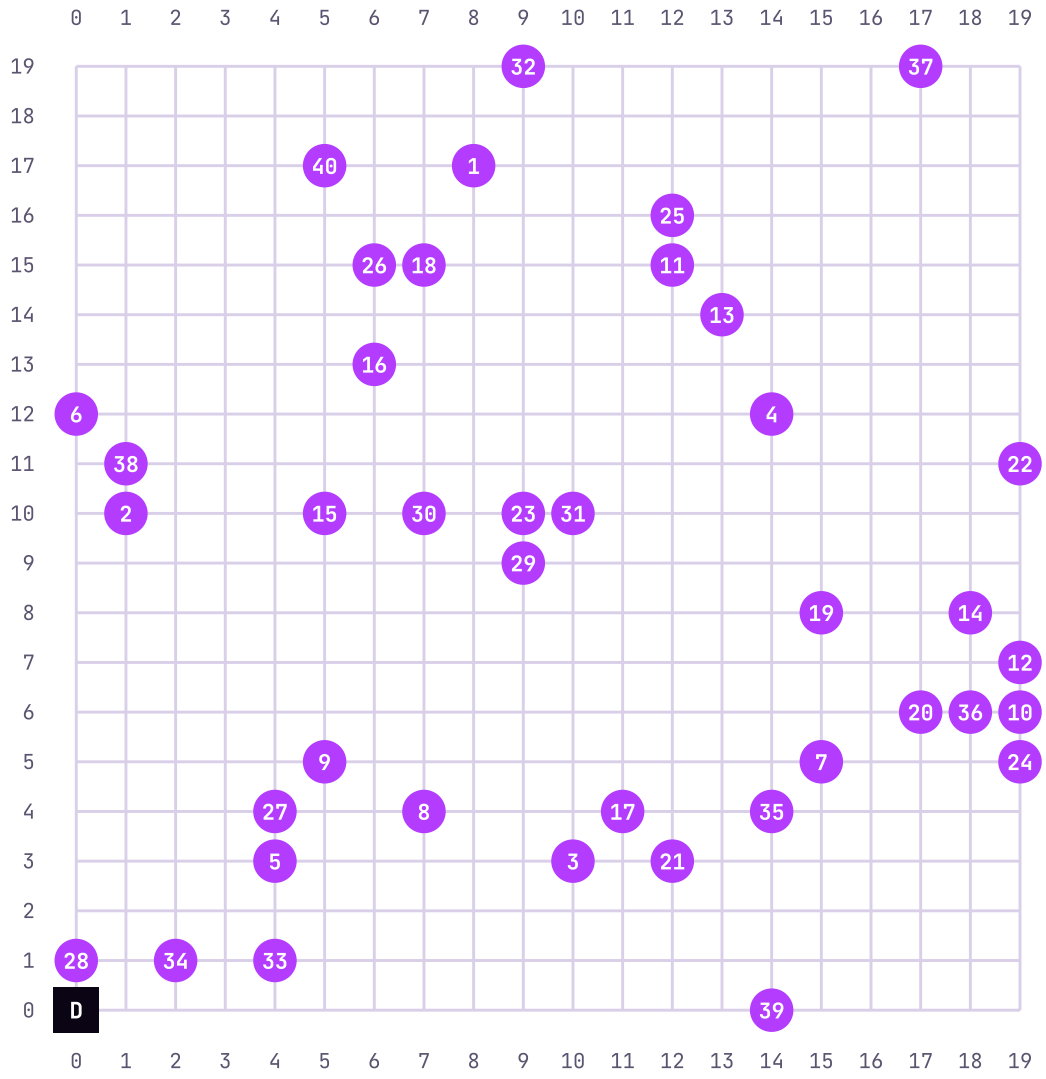
Nivel 1: el ejercicio de práctica

25 pedidos · 3 vehículos (V1, V2, V3) · depósito en (0, 0). Descargalo en la práctica de `https://dygd5d0drje17.cloudfront.net/simulador`.



Niveles 2 y 3: el ejercicio de práctica

40 pedidos · 5 vehículos (V1, V2, V3, V4, V5) · depósito en (0, 0). Descargalo en la práctica de `https://dygd5d0drje17.cloudfront.net/simulador`.



9 Entregas

Lo que desarrollan durante la jornada es **un programa**: recibe un dataset (mundo, pedidos y vehículos) y escribe un plan. El plan sirve solo para el dataset del que salió; el programa sirve para cualquiera. Por eso **se evalúa con datasets nuevos**, que no vieron antes.

	Práctica	Entrega
Datos	Los de práctica, públicos desde el principio	Uno nuevo en cada cierre, oculto hasta ese momento
Para qué	Probar y mejorar el programa	Sumar puntos
Cuenta para el puntaje	No	Sí

En cada cierre (la organización avisa cuándo)

1. Entrás a `https://dygd5d0drje17.cloudfront.net/simulador` con el **código de tu equipo** (`QD-XXXXX`).
2. **Descargás** el dataset del cierre: es nuevo, distinto del de práctica.
3. **Corrés tu programa** con ese dataset.
4. **Subís el plan** que generó. El costo sale del mismo simulador que la práctica.

Un plan armado para el dataset de práctica no sirve en la entrega: los pedidos son otros.

- Hay **unos minutos**: tu programa tiene que generar el plan sin pasos a mano.
- Podés entregar **varias veces** cada dataset mientras el cierre esté abierto.

> **Cada entrega reemplaza a la anterior: cuenta la última, aunque sea peor o inválida.** Probá tu programa con la práctica antes del cierre, y antes de volver a entregar revisá el costo que te mostró la entrega anterior.

10 Cómo se evalúa

Cada equipo termina con una nota sobre 100, comparada solo con su nivel:

De los 100 puntos…	Nivel 1	Nivel 2	Nivel 3
A. Calidad del plan (simulador)	50	35	30
J. Pitch (lo califica el jurado)	50	65	70

A la calcula el simulador con las entregas del corte final: en cada dataset, `100 × menor costo válido del nivel / tu costo` (inválido o sin entrega: 0), y se promedian.

El jurado califica **solo el pitch** (5 minutos de presentación y 3 de preguntas): no mira el código ni la solución. Pone una nota de 0 a 10 en cada uno de estos criterios, y J es su promedio:

Criterio	Qué mira
Toma de decisiones	Por qué eligieron su estrategia, con qué alternativas y con qué números.
Adaptabilidad	Qué hicieron cuando algo cambió o no funcionó.
Planificación	Cómo organizaron la jornada y priorizaron.
División de tareas	Quién hizo qué; que todo el equipo pueda explicar la solución.
Presentación	Claridad, tiempo y resultados concretos.

Un integrante del jurado le hace **una pregunta** a cada equipo.

Durante la jornada cambian los requisitos

Con cada evento te damos una guía nueva y se publica una versión nueva del simulador (`git pull`). Los equipos que diseñaron bien desde el principio solo agregan reglas; los que escribieron todo pegado al dataset de práctica tienen que reescribir. **Eso premia el diseño.**